



Editorial

Tasks, test data and solutions were prepared by: Fran Babić, Toni Brajko, Nikola Dmitrović, Marin Kupanovac, Ivan Jambrešić, Sofija Velkovska and Jakov Celin.

Implementation examples are given in attached source code files.

Problem Minesweeper

Prepared by: Marin Kupanovac

Required knowledge: matrices, loop statement (for), decision statement (if)

The problem could be solved in several ways, but we will describe the official solution. We solve the problem by entering the positions of the bombs (we mark them with the letter B or with -1 , the solution is identical) and then increasing by 1 all neighboring cells that are not bombs. Finding the positions of neighboring cells is the more demanding part of the problem, but they can be determined manually and written into an auxiliary list. After determining all values of the matrix, we output it as required in the problem statement. For details, see the official implementation.

Problem Dodatna

Prepared by: Jakov Celin

Required knowledge: sweep line, multiset

We first want to simplify the condition for holding extra classes from the problem statement. Extra classes can be held in some interval $[L, R]$ if for at least k students it holds that they are at school at second L and that they will remain at school at least until millisecond R .

This approach leads us to the idea for solving the problem. For each left boundary L , we will determine how long Jakov can hold extra classes **at most** if he decides to start them at second L , and then remember the maximum duration. Note that the only left boundaries relevant for obtaining the solution are those moments when some student arrives at school, because Jakov can start extra classes at the moment just before the current one if no student arrived at school at that time.

Let us fix the left boundary of holding extra classes to L . If we know all **right** boundaries of the intervals of students who are already at school at second L , then Jakov can hold extra classes until their k -th largest right boundary (if there are not enough students, i.e. fewer than k , then he cannot hold them at all).

In order to have, for each left boundary L , the list of all students who are at school at that second, we will use a sweep line algorithm and a structure that maintains the k largest right boundaries of students, as well as insertion and deletion of right boundaries from the structure. One such structure is a multiset.

We sort all events (student arrivals and departures) and process them in order. If we encounter the left boundary of some interval, we insert the student's right boundary into the multiset (if the multiset contains more than k right boundaries, we remove the smallest one), find the k -th largest right boundary, and store the solution if the multiset contains at least k right boundaries. If we encounter the right boundary of some interval, we remove it from the multiset if it is present and continue with the algorithm.

The total complexity of the algorithm is $O(n \log k)$, and for details see the official implementation.



Problem Natjecanje

Prepared by: Jakov Celin, Fran Babić, Fran Škarica

Required knowledge: BFS, blossom algorithm, maximum weighted matching in a graph

Note that the only information needed to obtain an arbitrary solution are the pairwise distances between packages and the distances of packages from the starting cell. All distances are found using the BFS algorithm, running it separately from each package. Now we can view the task as choosing some packages to carry together in pairs (deliver one, then the other, and return to the starting cell), and some individually. We solve the first subtask by trying all such combinations and taking the best one.

The remaining subtasks are solved using dynamic programming with bitmasks. Note that for a fixed bitmask *mask* we want to pair 2 packages (k^2 transitions) or deliver 1 package and return (k transitions). This approach solves the second subtask in complexity $O(2^k \cdot k^2)$. The third subtask is solved using the observation that every package in the final mask will be paired with some other package or with itself. From this it follows that when pairing 2 packages we can fix the first unpaired package in the mask and try to pair it with all remaining unpaired packages (k transitions per mask). This approach solves the third subtask in complexity $O(2^k \cdot k)$.

The solution of the last subtask requires knowledge of the [blossom](#) algorithm and its [modifications](#) for finding a maximum weight matching in a graph. This subtask is intended for preparation for the most demanding competitions, and other contest participants are not encouraged to solve this subtask afterward. We will nevertheless describe the solution, since the only demanding part of the problem is understanding and implementing the weighted blossom algorithm.

Instead of finding the minimum number of seconds needed to deliver all packages, we need to “turn the solution upside down” and initially deliver all packages unpaired. Then we construct a graph where packages i and j are connected by an edge whose weight equals the number of seconds Dino will save if he pairs them and delivers them together. The task then reduces to finding a maximum weight matching in a complete graph, because Dino’s goal is to save as much time as possible, i.e. to deliver all packages in as few seconds as possible. For details, see the official implementation and the previously attached links.

Problem Rastući

Prepared by: Ivan Jambrešić

Required knowledge: dynamic programming, two-pointers

We solve the problem using dynamic programming and reconstruction of the solution from the computed states. We will describe the official solution due to its simple implementation.

Let $l \leq r$. Define $suma(l, r)$ as the sum of values in the array between positions l and r , and $dp(l, r)$ as the maximum number of intervals into which we can partition the suffix of the array starting at position l , such that for the first interval $[l, R]$ of this partition it holds that $R \geq r$.

Then $dp(l, n) = 1$, and otherwise $dp(l, r) = \max(dp(l, r + 1), 1 + dp(r + 1, k))$, where k represents the smallest index such that $suma(l, r) \leq suma(r + 1, k)$. Such a k for fixed l and r will be denoted as $k(l, r)$. Naively finding $k(l, r)$ in complexity $O(n)$ for fixed l and r yields 70 points with correct reconstruction due to the excessive overall time complexity of $O(n^3)$.

It remains to efficiently find $k(l, r)$ for each $l \leq r$.

Note that for $l \geq 2$ it holds that $k(l - 1, r) \geq k(l, r)$. This property allows us to fix r and iterate over left boundaries l in decreasing order, increasing k as needed. For a fixed right boundary r we will perform $O(n)$ operations, so the total complexity of the algorithm is $O(n^2)$. For details, see the official implementation.



Task Tornjevi

Prepared by: Toni Brajko

Required knowledge: segment tree or RMQ

For each query we can directly simulate the construction of the towers. We go through the blocks from $i = l$ to $i = r$ and try to place each block on top of an existing tower whose top block has the opposite color. If such a tower exists, we attach the block to it. If it does not exist, we create a new tower whose initial block has the same color as the current one. After processing all blocks in the interval, the number of created towers is the required answer. The complexity of this approach is $O(nq)$, which is sufficient for solving the second subtask.

If the entire sequence contains at most 20 blue blocks, any color change can occur only at one of those positions. In the query (l, r) only the blue blocks within that interval are relevant, and between them lie segments of red blocks. The stacking idea remains the same as in the solution above, except that we skip large segments of red blocks. The complexity of this solution for the third subtask is $O(n + 20q)$.

Assign the value $+1$ to each red block and -1 to each blue block, and let p_i be the prefix sum of these values up to position i .

For a given query $[l, r]$, the difference $p_j - p_{i-1}$ for $l \leq i \leq j \leq r$ represents how many more red than blue blocks appear between positions i and j , or vice versa if the value is negative. The key claim is that the minimum number of towers needed for the interval $[l, r]$ equals

$$\max_{l \leq i \leq j \leq r} |p_j - p_{i-1}|.$$

Intuitively, as we scan the interval, any possible “surplus” of red over blue or blue over red corresponds to a moment when a new tower must be created. The worst such surplus corresponds to the largest positive subarray sum (if red is in surplus) or the largest negative subarray sum (if blue is in surplus). The minimum number of towers is therefore the largest absolute value that any such subarray sum can attain. Finding the subarray with the maximum absolute sum can be solved with a segment tree in $O(n + q \log n)$.

Furthermore, the expression above can be simplified to

$$\max_{l-1 \leq i \leq r} p_i - \min_{l-1 \leq i \leq r} p_{i-1},$$

which directly gives the answer to the query. This formula trivially holds when the index of the maximum is greater than the index of the minimum. Note that, in the opposite case, the largest absolute value is achieved by the subarray with the minimal (negative) sum, which we also obtain from the same formula. Therefore, it is not necessary to explicitly look for the subarray with the maximum absolute sum, as it suffices to find the maximum and minimum prefix sums separately.

To efficiently compute the minimum and maximum values, we can use a segment tree or an RMQ (range minimum query) table. The total complexity is $O(n + q \log n)$ or $O(n \log n + q)$, depending on the chosen implementation.